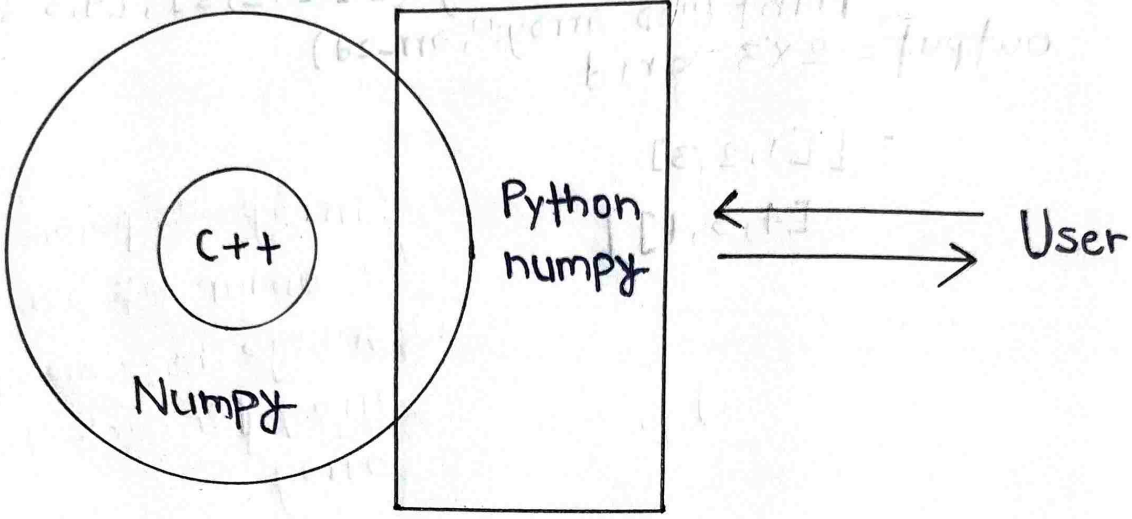
NUMPY (foundation)

PHASE - 1

(foundation, creation of array & method)



[1, 2, 3, 4, 5, 6] → Vector

[
[1, 2, 3],
[4, 5, 6]
] → Matrices

* In python "list" is in Numpy is "arrays".

→ **Arrays**

Python lists aren't enough for data science; you need to convert them into Numpy array using "np.array()".

- **1D Array** :- simple flat line array

code -

```
arr_1d = np.array()
```

example-

```
arr_1d = np.array([1, 2, 3])
print("1D array:", arr_1d)
```

output:- [1, 2, 3]

• **2D Array** :- Array nested inside an array (Rows & column).

code- `arr_2d = np.array([ ])`

example- `arr_2d = np.array([[1,2,3],[4,5,6]])`
`Print("2D array:", arr_2d)`

output = 2X3 grid

= [[1,2,3]
[4,5,6]]

Array ke parameter ke andar ek array chahiye hota mean array for each list/ array.

→ **Python Vs Numpy Arrays**

If we multiply a standard Python list by 2, "Python" just duplicates the items.

If we multiply a "Numpy" array by 2, it does element-wise multiplication the "2" number is called scaler

code-
```
Py_list = [1,2,3]
Print("python list Multiplication", Py_list *2)
```

output :- [1,2,3,1,2,3]

code-
```
np_array = np.array([1,2,3])
print("Python array multiplication", np_array *2)
```

output :- [2,4,6]

More example for py-list & np-array (speed test)

ex code =

```
import time
start = time.time()
py_list = [i*2 for i in range(1000000)]
print("In List operation time:", time.time() -
                                        start)
start = time.time()
np_array = np.arange(1000000) *2
print("In Numpy operation time:", time.time()
                                        - start)
```

List operation time = 0.029791819170124844

Numpy operation time = 0.0019657118462383

(2) Ones:

ex code =

```
ones = np.ones((2,3))        (row, column)
print("one array: \n", ones)
```

output = [[1,1,1]
[1,1,1]]

(3) Full:

ex code =

```
full = np.full((2,2), 7)      (Rows, column, value)
print("full array: \n", full)
```

output = [[7,7]
[7,7]]

(4) Random:

fills the shape with random decimal values between (0,1)

ex code =

```
random = np.random.random((2,3))
print("random array: \n", random)
```

output = [[0.49377362, 0.881602, 0.9490922]
[0.14355, 0.3722628, 0.22087662]]

* speed : Python list ~ 0.031 sec
Numpy ~ 0.0009 sec (much faster)

→ **Creating Arrays from scratch**

Typing out massive lists manually is impossible, so "Numpy" gives you **automated methods**

(1) Zeros:

ex code =

```
zeros=np.zeros((3,4))
Print("zeros array : \n", zeros)
```

Create matrices of 3 row & 4 column

output :-
```
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]
```

(2) Ones:

ex code =

```
ones =np.ones((2,3))
print("one array: \n",ones)
```

(row, column)

→ create a grid filled one

output =
```
[[1. 1. 1.]
 [1. 1. 1.]]
```

(3) Full:

ex code =

```
Full=np.full((2,2), 7)
print("full array : \n", full)
```

(Rows, column, value)

output =
```
[[7. 7.]
 [7. 7.]]
```

(4) Random:

fills the shape with random decimal values

ex code =

```
random=np.random.random((2,3))
print("random array : \n",random)
```

output =
```
[[0.49377362, 0.881602 , 0.9490922]
 [0.14355  , 0.3722628 , 0.2208762]]
```

(5) sequence (arange):

Range

```
sequence = np.arange(0, 10, 2)
print("random array :\n", sequence
```

ex code =

output = [0, 2, 4, 6, 8]

→ **Vector, Matrix, Tensor** (Data science Vocabulary)

(1) Vector (1D)

ex code

```
vector = np.array([1, 2, 3])
print("vector:" vector)
```

output - [1, 2, 3]

*In python range is always a non-enclusive boundary mean - if we give (1 to 7) it goes only till 6

(2) Matrix (2D) : Rows & column

ex code

```
matrix = np.array([[1, 2, 3],
                   [4, 5, 6]])
print("Matrix :", matrix)
```

there is enter for next line for look arrange & good

output - [[1, 2, 3]
[4, 5, 6]]

(3) Tensor (3D+): beyond 2D visualizing anything become difficult, we just call them "Tensor"

Array inside array inside array.

ex code =

```
tensor = np.array([[[1, 2], [3, 4]],
                   [[5, 6], [7, 8]]])
print("Tensor :", tensor)
```

output - [[1, 2]
[3, 4]]

[[5, 6]
[7, 8]]

→ Array Properties

When we load up new data, we use these properties to understand exactly what we are working with

(1) arr.shape : Tell us exact rows & column

ex code =

```
arr = np.array ([[1, 2, 3],
                 [4, 5, 6]])
Print ("shape", arr.shape)
```

output:- shape (2, 3)

(2) arr.ndim : number of dimension.

ex code =

```
arr = np.array ([[1, 2, 3],
                 [4, 5, 6]])
Print ("Dimension", arr.ndim)
```

output :- Dimension 2

(3) arr.size :- total elements or total no. of items inside.

ex code =

```
arr = np.array ([[1, 2, 3],
                 [4, 5, 6]])
Print ("size", arr.size
```

output :- size 6

(4) arr.dtype :- Tell the data type

ex code =

```
arr = np.array ([[1, 2, 3],
                 [4, 5, 6]])
print ("DType", arr.dtype
```

output = D Type int64

→ Array Reshaping

We will constantly need to change the structure of the data that the use.

(1) .reshape : Bends the flat 1D line into a 2D matrix

ex code =

```
arr = np.arange(12)
reshaped = arr.reshape((3,4))
Print("\n Reshaped array", reshaped)
```

Output :- Reshaped Array [[0, 1, 2, 3]
[4, 5, 6, 7]
[8, 9, 10, 11]]

(2) .T (Transpose): Diagonally flips our matrix, turning rows into column (a 3X4 grid becomes a 4X3 grid.)

ex code =

```
arr = np.arange(12)
reshape = arr.reshape((3,4))
transpose = reshaped.T
Print("\n Transposed array", transpose
```

output :- Transposed array [[0, 4, 5]
[1, 5, 9]
[2, 6, 10]
[3, 7, 11]]

(3) flatten() :- returns copy (.flattend)

ex code =

```
arr = np.arrange(12)
reshape = arr.reshape((3,4))
flattened = reshaped.flatten()
Print("\n flattened array", flattened)
```

Output = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]

(4).ravel() → returns view (Modifies original)

ex code =

```
arr = np.arrange(12)
reshaped = arr.reshape((3,4))
raveled = reshaped.ravel()
print("\n raveled array", raveled)
```

output = raveled array [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]

PHASE-2

(Operations on Numpy array)

at first line
import numpy as np

→ Indexing & slicing

(1) Basic slicing :- ex:- `arr[2:7]` ; grab elements starting from position 2 up to position 7.

start position ← 2 : 7 → end position

or elements from index 2 to 6

ex code =

```
arr = np.array([1,2,3,4,5,6,7,8,9,10])
print("Basic slicing", arr[2:7])
```

output :- Basic slicing [3,4,5,6,7]

(2) With steps:- ex:- `arr[1:8:2]` ; grabs element from position 1 to 8, jumping every 2nd step.

start position ← 1 : 8 : 2 → steps to jump/skip

end position ← 8

ex code =

```
arr = np.array([1,2,3,4,5,6,7,8,9,10])
print("With step", arr[1:8:2]
```

output :- With step [2,4,6,8]

(3) Negative Indexing :- ex:- `arr[-3]` ; starts counting from end, grabbing the 3rd value from the last

3rd value from last ← -3

ex code =

```
arr = np.array([1,2,3,4,5,6,7,8,9,10])
print("Negative indexing", arr[-3])
```

output :- Negative indexing 8

(4) Specific element (In 2D Matrices) :-

ex:- `arr_2d[1, 2]` ; It gives us the item in the 2nd row (index 1) & 3rd column (index 2)

Row ← 1 , 2 → column

ex code :-

```
arr_2d = np.array([[1,2,3],
                   [4,5,6],
                   [7,8,9]])
print("specific element", arr_2d[1,2])
```

output - specific element 6

Row & column counting starts from zero

• Entrie Row & Entire column :-

Entire Row ex:- `arr_2d[1, :]` ; It gives us the complete 2nd row.

or

`arr_2d[1]`

1 → Row ; : → give me all column

Entire column ex:- `arr_2d[:, 1]` ; It gives us the 2nd column

: → give us all rows ; 1 → column

*use colon (:) ; in this situation for give me all row but only targets the 2nd column.

ex code =

```
arr_2d = np.array([[1,2,3]
                   [4,5,6]
                   [7,8,9]])
print("Entire row:", arr_2d[1])
print("Entire column:", arr_2d[:,2])
```

output = Entire row: [4,5,6]

Entire column: [2,5,8]

→ Sorting

Using this we can easily organise scrambled data.

(1) 1D sort :- `np.sort(unsorted)` ; take a unsorted/scrambled array & returns sorted array

ex code =

```
unsorted = np.array([3, 1, 4, 1, 5, 9, 2, 6])
print("Sorted Array", np.sort(unsorted))
```

output = sorted array [1, 1, 2, 3, 4, 5, 6, 9]

(2) 2D sort by Axis :- `np.sort(arr_2d_unsorted, axis=0)`

It can also 1 but Mostly we use 1 for tabular data

for tubular data (like a spreadsheet), we use axes.

It sort the data column by column, from top to bottom.

ex code =

```
arr_2d_unsorted = np.array([[3, 1], [1, 2], [2, 3]])
print("sorted 2D array by column",
np.sort(arr_2d_unsorted, axis=0))
```

output = sorted 2D array by column [[1,1]
[2,2]
[3,3]]

→ Filter

We don't need traditional "for loops" to find specific numbers. Numpy let us pass math directly into the brackets.

ex - `number [number % 2 == 0]` ; instantly returns an array of only the even numbers.

by like number/2

ex code =

```
numbers = np.array([1,2,3,4,5,6,7,8,9,10])
even_number = number [number % 2 == 0]
print("Even numbers", even_number)
```

→ **Filter with Mask** :- ex:- `mask = number > 5` ; It created a Mask. It doesn't store the number ; it stores map of True/False (ex - Is this no. greater than 5).

When we run "`number [mask]`" Numpy applies that True/False map & only prints the numbers where the condition was true.

ex code =

```
mask = number > 5
print ("Number greater than 5", numbers[mask])
```

output = Number greater than 5 [6, 7, 8, 9, 10]

→ **Fancy indexing vs np.where()**

One of the most powerful tool in Data science.

• **Basic use** : ex- `np.where(numbers > 5)` , It will return the exact index positions of the elements that pass the test.

ex code =

```
arr = np.array([1,2,3,4,5,6,7,8,9,10])
indices = [0,2,4]
print (arr[indices])
where_result = np.where(arr > 5)
print (where_result)
print("NP Where"), arr[where_result])
```

output = [1, 3, 5]
(array([5, 6, 7, 8, 9]),)
NP where : [6, 7, 8, 9, 10]

• **If/Else Manipulation:** ex:- `np.where(number > 5, "true", "False")`

IF the number greater than 5, replace with "true". **ELSE**, replace it with "false". This completely transform our array based on a condition without writing a single loop.

ex code =

```
numbers = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
condition_array = np.where(numbers > 5, "true",
                           "False")
print(condition_array)
```

output = ['false', 'false', 'false', 'false', 'false', 'true', 'true', 'true', 'true', 'true']

→ Combining Arrays/Adding

• * Never use arr1 + arr2 to combine; Numpy just add two array as mathmatical addition. ex - (arr1 + arr2)

ex:- for Combine or Add

```
combined = np.concatenate((arr1, arr2))
```

ex code =

```
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])

combined = np.concatenate((arr1, arr2))
print(combined)
```

output = [1, 2, 3, 4, 5, 6]

→ Array compatibility / shape compatibility

for complex , Data science operation , arrays often need to be the exact same size . We can easily verify them with `a.shape == b.shape` which return True/False

ex code =

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6, 7])
c = np.array([7, 8, 9])
print("compatibility shapes", a.shape == b.shape)
```

output = Compatibility shapes False

→ stacking (some unique operation)

It is not name on numpy It is unique operation we just name it on own way.

When adding new data to a matrix, we can stack it vertically or horizontally

• `np.vstack((original, new_row))` : vertically stack data , perfectly slotting a new row underneath our matrix

ex code =

```
original = np.array([[1,2], [3,4]])
new_row = np.array([[5,6]])
with_new_row = np.vstack((original, new_row))
print(with_new_row)
```

output =
```
[[1,2]
 [3,4]
 [5,6]]
```

• hstack joins arrays horizontally mean column wise

• vstack always add a row

row goes always left to right

• **np.hstack((original, new_col))**:

Horizontally stacks data, adding a new column to the right side

ex code -

```
original = np.array([[1,2],[3,4]])
new_row = np.array([[5,6]])
new_col = np.array([[7],[8]])
with_new_col = np.hstack((original, new_col))
print("with new column", with_new_col)
```

output :- with new column [[1, 2, 7]
[3, 4, 8]]

→ **Deleting Data** ex- `np.delete(arr, 2)` ; This will delete the element sitting at index 2

ex code :

```
arr = np.array([1,2,3,4,5])
deleted = np.delete(arr, 2)
print("Array After deletion:", deleted)
```

output = Array after deletion : [1 2 4 5]

* key ideas
- Rows = horizontally (left ↔ Right)
- column = vertically (top ↔ bottom)
- hstack() → work column wise
- vstack() → work row wise

axis terms
- axis = 0
  ↳ downwards (row)
- axis = 1
  ↳ sideways (column)

PHASE -3

(PRACTICE WITH REAL WORLD DATA)

#Data structure : [restaurant_id , 2021, 2022, 2023, 2024]

```
Sales_data = np.array([
    [1 , 150000 , 180000, 220000, 250000] , #Paradise Biryani
    [2 , 120000, 140000 , 160000 , 190000] , #Beijing Bites
    [3 , 200000 , 230000 , 260000 , 300000] , # Pizza hub
    [4, 180000 , 210000 , 240000 , 270000] , #Burger Point
    [5, 160000 , 185000 , 205000 , 230000] , #chai point
    ])
```

↳ We are using this "sales_data" in all topics in this phase . this example of sales-data will be same for all examples.

→ **The sales Dataset & Advanced Slicing**

(1) The shape (.shape) - ex = `sales_data.shape` ; It tells exact rows & column in matrix.

ex code =

```
sales_data = np.array ([ .... ])  #already given on top
print ("In sales data shape ", sales_data.shape)
```

output = sales data shape (5, 5) (rows, columns)

(2) Slicing out the "Noise" : of we want specific data mean specific rows or column. then we can use this technique

ex = `sales_data[:, 1:]`

start — end — start — end

- `:` mean all rows (we can the range row start & end)
- `1:` mean start from index 1 (mean 2nd column) to the end column/index.

ex code =

```
sales_data = np.array([ ... ])  # already given
print("In sample data for 1st 3 restaurant",
      sales_data[:, 1:])
```

output = Sample data for 1st 3 restaurant:

```
[[150000, 180000, 220000, 250000]
 [120000, 140000, 160000, 190000]
 [200000, 230000, 260000, 300000]
 [180000, 210000, 240000, 270000]]
```

* we can also select any row or column. using these.
* sales_data.[row, column]

sales_data[row, column]
ex: : = mean all
3:1 = range slicing
[0, 3, 4] = specific column

→ Sum / Min / Max / Avg / cumsum (cumalative sum)

(1) np.sum :

ex =

```
print(np.sum(sales_data, axis=0))
```

* for yearly total code =

```
sales_data = np.array([ ... ])
yearly_total = np.sum(sales_data
    [:, 1:], axis = 0
print(yearly_total)
```

Here we can decide any specific row, row

(2) np.min : for Minimum value

Code =
```
sales_data = np.array([...])
min_sales = np.min(sales_data[:, 1:], axis=1)
print(min_sales)
```

(3) np.max: for Maximum value

Code =
```
sales_data = np.array([...])
max_sales = np.max(sales_data[:, 1:], axis=0)
print(max_sales)
```

(4) np.mean/Avg: To find Average

code =
```
sales_data = np.array([...])
Avg_sales = np.mean(sales_data[:, 1:], axis=1)
print(Avg_sales)
```

(5) cumsum/np.cumsum :- "cumulative sum" of element, meaning each value is the sum of all previous elem. including itself

code =
```
sales_data = np.array([...])
cumsum = np.cumsum(sales_data
[:, 1:], axis=1)
print(cumsum)
```

* To Plot a Image Graph of cumsum using matplot library

- Matplot library : Python library to create graphs & visualize data (like charts, Plots etc.)

To Import Matplot library :-

```
import matplotlib.pyplot as plt
```

ex code =

```
import numpy as np
import matplotlib.pyplot as plt
sales_data = np.array([...])
cumsum = np.cumsum(sales_data[:,1:],
 axis = 1)
print(cumsum)
plt.figure(figsize=(8,6))
plt.plot(np.mean(cumsum, axis=0))
plt.title("Avg cumulative sales across all
                restaurent")
plt.xlabel("Years")
plt.ylabel("sales")
plt.grid(True)
plt.show()
```

output = We get a Visuals Image graph

→ Vectors, Dot Products & AI Database or AI connection of Vector

(1) Vector : Its just a list of number that represents data (like features of something)

V1 = [1, 2, 3, 4, 5]

V2 = [6, 7, 8, 9, 10]

In code =

```
Vector 1 = np.array([1, 2, 3, 4, 5])
Vector 2 = np.array([6, 7, 8, 9, 10])
```

• **Vector Addition** : to add vectors

ex code =

```
vector1 = np.array([1, 2, 3, 4, 5])
vector 2 = np.array([6, 7, 8, 9, 10])
print("Vector addition:",
      vector1 + vector 2)
```

• **Vector Multiplication** :

```
vector1 = np.array([1, 2, 3, 4, 5])
vector2 = np.array([6, 7, 8, 9, 10])
print("Dot Product", vector1 * vector2)
```

(2) Dot Product

Dot product = multiply matching elements + add them all.

In AI/ML it is used for :

- compore two vectors (data)
- Recommendation system
- search engine
- Projection
- Angle between vectors etc.

ex code :

```
vector1 = np.array([1, 2, 3, 4, 5])
vector2 = np.array([6, 7, 8, 9, 10])
print("Dot product", np.dot(vector1,
      vector 2)
```

• np.arccos(): It means inverse cosine, it gives the angle from a cosine value.

ex code =

```
vector1 = np.array([1,2,3,4,5])
vector2 = np.array([6,7,8,9,10])
angle = np.arccos(np.dot(vector1, vector2)/
        (np.linalg.norm(vector1) * np.
        linalg.norm(vector2)))
```

(3) The AI connection :- In Modern AI, dot product & vector Maths are used in vector databases like pinecone to Measure similarity between data.

By normalizing vectors "(np.linalng.norm)" & finding angle using "np.arccos", AI calculates cosine similarity to determine how closely two data points are related.

→ **Vectorisation** : It make normal python code compatible with Numpy arrays so that it performs the operation individually on every single item, rather than treating the array as one giant project.

* for making uppercase :-

ex code =

```
restaurent_types = np.array(['biryani',
'chinese', 'pizza', 'burger', 'cafe'
upper_types = np.char.upper(restaurent_
                types)
print(upper_types)
```

output = ['BIRYANI', 'CHINESE', 'PIZZA', 'BURGER', 'CAFE']

for vactorisation :-

ex code =

```
restaurent_types = np.array(['biryani',
'chinese', 'pizza', 'burger', 'cafe'])
vectorized_upper = np.vectorize(str.upper)
print("vectorized upper", vectorized_upper
        (restaurent_types))
```

output = vectorized upper ['BIRYANI', 'PIZZA', 'BURGER', 'CAFE']

→ Broadcasting:

Broadcasting is when a single value (like 1e) automatically applies to every element of an array during an operation.

ex:- If we want to calculat Monthly Avg of yearly restaurants.

solution- we just divide Matrix by 12

ex code

```
sales_data = np.array ([...])
Monthly_avg = sales_data[ : , 1: ] /12
print (Monthly_avg
```

Each element of the Matrix is divided by 12.

Don't just blindly Memorize "axis = 1". Always Imagine the Matrix. If the data we want to calculate moves horizontally across a row, it's `axis = 1`. if it moves vertically down a column, it's `axis = 0`

PHASE - 4

(STORING IMAGE IN MATRIX & CONVERT IMAGE IN DARK MODE)

→ Saving Arrays to computer

Data we create in Numpy stays in Temporary memory (RAM), so we must save it to our computer to keep it safe

`np.save()` with `.npy`

mean

code =

```
np.save ('array1.npy', array1)
```

Ex code =

```
array1 = np.save([ [1,2,3], [4,5,6] ])
array2 = np.random.rand(3,3)
array3 = np.zeros((4,4))
#To save these we do this
np.save('array1.npy', array1)
np.save('array2.npy', array2)
np.save('array3.npy', array3)
```

output = #file get save as

array1.npy

array2.npy

array3.npy

Notice ".npy" Numpy uses this special, highly optimized file extension specifically for storing its arrays.

→ Loading Arrays Back into Memory

We can pull previously saved ".npy" data back into our Jupyter Notebook to continue working at any time.

`np.load()` with ".npy"

ex = `np.load('array1.npy')`

ex Code =

```
loaded-array1 = np.load('array1.npy')
print (loaded-array1)
```

When we load the file, it works just like a normal Numpy array & we can use it normally

→ Image as Matrices & Modern AI

Every digital image we see on a screen is actually just a massive multi-dimensional Numpy matrix of Pixels containing Red, Green, & Blue (RGB) number values.

- **The AI Connection:** Modern AI image generators (like chatgpt or Flux) do not "paint" like humans. Instead, they generate image by Mathematically predicting exactly what number to put into these Numpy pixel matrices.

→ Visualizing Images with Matplotlib

Since images are just Numpy matrices, we can load them & display them on our screen using Matplotlib.

(1) first we need to load the actual image like here we have: "numpy-logo.npy".

ex code =

```
try:
    logo = np.load('numpy-logo.npy')
    #Display                    ↳ loaded image
    plt.figure(figsize=(10,5))
    plt.subplot(121)
    plt.imshow(logo)
    plt.title("Numpy logo")
    plt.grid(False)
```

→ "Dark Mode" hack to (Inverting colors)

Because an image is just a matrix of numbers, we can alter the image by doing simple element-wise math on it!

We don't need "for-loops" to change pixels; Numpy can do it directly.

• The Hack :- code:- `dark_logo = 1 - logo` , By substracting the entire pixels matrix from 1, Numpy

instanly inverts all the colors seamless, perfectly turning the white background to black.

ex code :-

```
import numpy as np
import matplotlib.pyplot as plt

try:
    logo = np.load('numpy-logo.npy')
    #Display
    plt.figure(figsize = (10,5))
    plt.subplot(121)
    plt.imshow(logo)
    plt.title("Numpy logo")
    plt.grid(False)

    dark_logo = 1- logo
    plt.subplot(122)
    plt.imshow(dark_logo)
    plt.title("Numpy Dark logo")
    plt.grid(False)

except FileNotFoundError:
    print("Numpy logo file not found")
```